

# Building an Ultrasonic Radar using Arduino and Processing

This project combines an Arduino-controlled ultrasonic sensor with a servo motor to scan for and measure the distance to nearby objects. The collected data is then visualized on your computer as a live, graphical radar screen using the Processing environment.

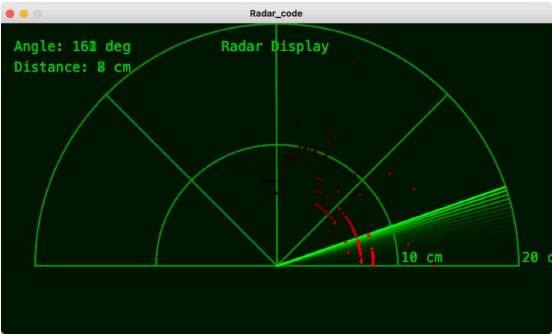
Jun 29, 2025 • 152347 views • 76 respects

- GPL3+



Monitoring

Environmental Sensing



## Devices & Components



1 Arduino Uno  
Rev3



1

Ultrasonic Sensor  
- HC-SR04



1

Servo Motor SG90  
180 degree



## Software & Tools

1

Processing 3



Arduino IDE



## Project description

This project details the construction of a simple yet effective 2D ultrasonic radar system. It serves as a classic introductory project that seamlessly bridges the gap between physical hardware and software-based data visualization. The system uses an Arduino microcontroller to control a sweeping ultrasonic sensor, which detects the presence and distance of objects in a 180-degree field of view. This spatial data is then transmitted to a computer and rendered in real-time as an intuitive, graphical radar display using the Processing development environment.

The final output is a visually satisfying and functional device that mimics the principles of real-world sonar and radar systems, providing a tangible representation of its surroundings.

- ◆ To construct a physical scanning mechanism using a micro servo motor and an HC-SR04 ultrasonic sensor.
- ◆ To program an Arduino board to control the servo motor's sweep and trigger the ultrasonic sensor for distance measurements.
- ◆ To establish reliable serial communication between the

Arduino and a host computer to transmit sensor data (angle and distance).

- ◆ To develop a Graphical User Interface (GUI) in the Processing environment to visualize the incoming data as a radar screen.
- ◆ To integrate the hardware and software components into a fully functional object detection and visualization system.

#### Hardware:

- ◆ Arduino UNO (or compatible board): The central microcontroller that executes the control logic.
- ◆ HC-SR04 Ultrasonic Distance Sensor: The "eyes" of the project, used to measure distances via sonar.
- ◆ SG90 Micro Servo Motor: An actuator that physically rotates the ultrasonic sensor.
- ◆ Breadboard and Jumper Wires: For creating and managing electrical connections without soldering.
- ◆ USB A-to-B Cable: To power the Arduino and establish serial communication with the computer.

#### Software:

- ◆ **Arduino IDE:** Used to write, compile, and upload the control code to the Arduino board.
- ◆ **Processing Development Environment:** A flexible software sketchbook used to write the code for the radar visualization on the computer.

### Methodology:

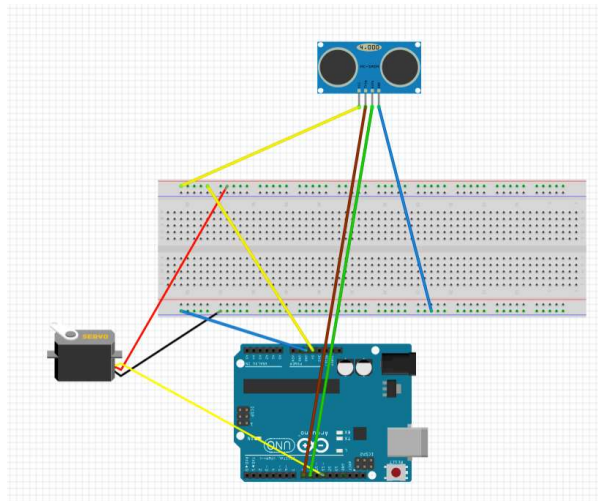
- ◆ **The operation of the radar is a coordinated effort between the physical components and the two software environments.**
- ◆ **Scanning Motion:** The Arduino board sends a precise signal to the servo motor, commanding it to rotate in small, incremental steps from 0 to 180 degrees and then back again.
- ◆ **Distance Measurement:** At each angular step, the Arduino triggers the HC-SR04 ultrasonic sensor. The sensor emits a high-frequency sound pulse and measures the time it takes for the echo to bounce back from an object.
- ◆ **Data Calculation:** The distance to the object is calculated within the Arduino sketch using the formula:

$$\text{Distance} = (\text{Speed of Sound} \times \text{Time}) / 2$$

The result is a pair of data points: the current angle of the servo and the calculated distance.

**Serial Communication:** The Arduino formats this angle and distance data into a string and transmits it to the computer via the USB serial port.

**Data Visualization:** On the computer, a sketch running in the Processing environment continuously listens to the serial port. When it receives the data string from the Arduino, it parses the angle and distance values. Using trigonometry, it converts these polar coordinates into Cartesian (x, y) points to plot on the screen, creating a visual map of any detected objects relative to the sensor's position.



Simple diagram

This is just part of a bigger project, but I learned a lot doing it. Have a great day.

## Code

### arduino radar code



```
1  #include <Servo.h>
2
3  // --- Pin Definitions ---
4  const int SERVO_PIN = 11;
5  const int TRIG_PIN = 8;
6  const int ECHO_PIN = 9;
7
8  // --- Constants for Servo -
   --
9  const int MIN_ANGLE = 0;
10 const int MAX_ANGLE = 180;
11 const int ANGLE_STEP = 1;
12 const int SWEEP_DELAY = 15;
   // Delay between servo
   movements in milliseconds
```

### Processing radar code



```
1 | # Processing (Python) code  
   | for a Radar Display  
2 | # This script reads angle  
   | and distance data from an  
   | Arduino over the serial port  
3 | # and visualizes it as a  
   | classic radar screen.  
4 | #  
5 | # MODIFIED to use a  
   | rectangular, half-screen  
   | layout.  
6 |  
7 | add_library('serial')  
8 |  
9 | # --- Global Variables ---
```

## Comments



Only logged in users can leave comments

[LOGIN](#)

**R** **rlmariano**

7 days ago



AMAZING! I HAD TO CORRECT THE CODE. "PROCESSING" DIDN'T ACCEPT PYTHON AND I HAD TO DO IT IN JAVA.

**A** **alem-511**

a month ago

